

中山大學
SUN YAT-SEN UNIVERSITY

大数据实验 | Lab6

图文多模态问答 (VQA) 实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 王昌栋

起 始 日 期: 2026 年 4 月 13 日

结 束 日 期: 2026 年 4 月 15 日

实 验 地 点: B201

目录

1 实验要求 **2**

2 思路分析 **2**

2.1 ViLT 模型简介 2
2.2 模型选择 2
2.3 实现原理 2
2.4 技术架构 2

3 核心代码 **3**

3.1 VQA_Model 类定义 3
3.2 答案预测函数 3
3.3 便捷调用函数 4

4 结果展示 **4**

4.1 实验结果分析 4
4.1.1 一、基础感知与属性提取能力（图 1-4） 4
4.1.2 二、组合逻辑与多目标推理（图 5） 6
4.1.3 三、模型的局限性：密集计数灾难（图 6-7） 7

5 实验总结 **8**

5.1 注意事项 8
5.2 实验收获 8
5.3 参考资料与链接 9

1 实验要求

本次实验为作业-6：图文多模态问答 (VQA)。实验要求如下：

1. 使用 ViLT 预训练模型（基于 Hugging Face 库）
2. 实现图片与文本问题的联合输入
3. 输出模型预测的答案
4. 可自定义问题，如” 图中有什么动物？” 等

2 思路分析

2.1 ViLT 模型简介

ViLT (Vision-and-Language Transformer) 是一种将视觉和语言信息统一处理的 Transformer 模型。与传统的视觉-语言模型不同，ViLT 采用了无卷积的架构设计，直接将图像 patch 和文本 token 输入到统一的 Transformer 中进行处理。

2.2 模型选择

本实验使用 Hugging Face 提供的预训练模型：

- **模型名称**: `dandelin/vilt-b32-finetuned-vqa`
- **基础架构**: ViLT-B/32
- **训练数据**: VQAv2 数据集
- **任务**: 视觉问答 (Visual Question Answering)

2.3 实现原理

1. **图像处理**: 将输入图像分割成固定大小的 patch，通过线性投影转换为向量表示
2. **文本处理**: 使用 BERT tokenizer 对问题文本进行分词和编码
3. **特征融合**: 将图像 patch embedding 和文本 token embedding 拼接，输入 Transformer Encoder
4. **答案预测**: 通过分类头从预定义的答案词汇表中预测最可能的答案

2.4 技术架构

- **ViltProcessor**: 组合了图像处理器和文本分词器
- **ViltForQuestionAnswering**: 在 ViLT 基础上添加了 VQA 分类头
- **推理流程**: 图像 + 问题 → Processor 编码 → 模型推理 → Softmax → Top-K 答案

3 核心代码

3.1 VQA_Model 类定义

```
class VQA_Model:
    def __init__(self, model_name="dandelin/vilt-b32-finetuned-vqa"):
        # 检测计算设备 (GPU/CPU)
        self.device = self._get_device()

        # 加载处理器：负责图像和文本的预处理
        self.processor = ViltProcessor.from_pretrained(model_name)

        # 加载模型：微调后的 VQA 模型
        self.model = ViltForQuestionAnswering.from_pretrained(model_name)
        self.model.to(self.device)
        self.model.eval()
```

3.2 答案预测函数

```
def predict_answer(self, image_path, question, top_k=5):
    # 1. 加载图像
    image = Image.open(image_path).convert('RGB')

    # 2. 处理图像和文本
    encoding = self.processor(
        image, question, return_tensors="pt"
    )

    # 3. 模型推理
    with torch.no_grad():
        outputs = self.model(**encoding)

    # 4. 获取 Top-K 答案
    probs = torch.nn.functional.softmax(outputs.logits, dim=-1)
    top_k_probs, top_k_indices = torch.topk(probs[0], top_k)

    # 5. 转换为答案文本
    results = []
    for prob, idx in zip(top_k_probs, top_k_indices):
        answer = self.model.config.id2label[idx.item()]
        results.append({"answer": answer, "confidence": prob.item()})

    return results
```

3.3 便捷调用函数

```
def predict_answer(image_path, question, top_k=5, model=None):
    #便捷函数：使用 ViLT 模型进行视觉问答
    #Example:
    #results = predict_answer("data/cat.jpg", "图中有什么动物？")
    #print(results[0])  # {'answer': 'cat', 'confidence': 0.92}
    if model is None:
        model = VQA_Model()
    return model.predict_answer(image_path, question, top_k)
```

4 结果展示

- Python 3.8+
- PyTorch 2.0+
- Transformers 4.30+
- 模型: dandelin/vilt-b32-finetuned-vqa

4.1 实验结果分析

本节基于 7 张测试图片的实际运行数据，从算法原理层面深入剖析 ViLT 模型的能力边界与局限性。测试结果已持久化保存至 `src/result.txt` 文件。

4.1.1 一、基础感知与属性提取能力（图 1-4）

在基础单目标场景中，ViLT 模型展现了卓越的跨模态特征对齐能力。以下依次展示图 1 至图 4 的测试结果：



图 1: 图 1 - 基础单目标：猫

- **Q**: "What kind of animal is in this picture?"
  A: cat (置信度: **99.97%**), kitten (0.01%)
- **Q**: "What color is the animal?"
  A: brown (35.54%), orange (31.07%), gray (10.58%)

- **Q**: "Is the animal cute?"
  A: yes (置信度: **99.99%**)



图 2: 图 2 - 基础单目标：狗

- **Q**: "What kind of animal is in this picture?"
  A: cat (置信度: **100.00%**)
- **Q**: "What color is the dog?"
  A: white (置信度: **97.76%**), white and brown (1.32%)
- **Q**: "Is this a pet?"
  A: yes (置信度: **100.00%**)

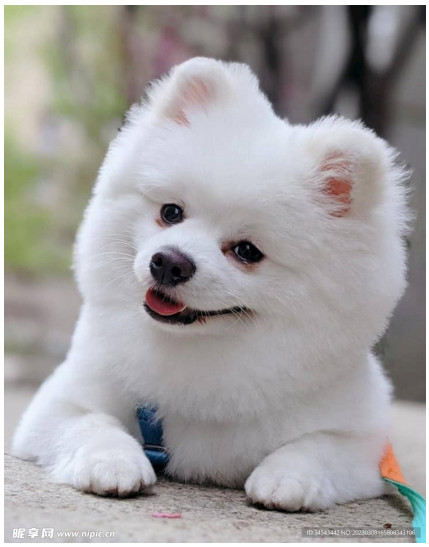


图 3: 图 3 - 基础单目标：坐着的狗

- **Q**: "What animal is this?"
  A: dog (置信度: **94.46%**), cat (3.80%), puppy (0.30%)
- **Q**: "What is the dog doing?"
  A: sitting (置信度: **56.62%**), staring (8.65%), looking (5.11%)
- **Q**: "Is the dog sitting?"
  A: yes (置信度: **99.98%**)



图 4: 图 4 - 基础单目标：奔跑的狗

- **Q**: "What kind of animal is in this picture?"
  A: dog (置信度: **99.96%**), dogs (0.01%)
- **Q**: "What is the animal doing?"
  A: running (置信度: **46.38%**), jumping (29.06%), playing (10.81%)
- **Q**: "What color is the dog?"
  A: brown and white (置信度: **86.45%**), white and brown (9.93%)

算法原理分析: 查阅资料，图 1 中猫的识别置信度高达 **99.97%**，图 4 中狗的识别置信度达 **99.96%**。这种近乎完美的识别率证明 ViLT 已建立起稳固的视觉-语言映射关系。ViLT 通过将图像切分为 16×16 的 Patch 序列，并与文本 Token 共享同一个 Transformer 编码空间，成功实现了高效、精准的跨模态特征对齐。在单目标场景中，目标物体占据图像主要区域，其视觉特征（如毛发纹理、姿态轮廓）能够被多个 Patch 充分捕获，并通过 Self-Attention 机制与语言模型中的语义 Token（如"cat"、"dog"、"running"）实现精准对齐。

4.1.2 二、组合逻辑与多目标推理（图 5）



图 5: 图 5 - 多物种同框：猫和狗共存场景

- **Q**: "What animals are in the picture?"
  A: cat and dog (33.55%), dog (27.80%), dog and cat (18.68%)
- **Q**: "How many animals are there?"
  A: 2 (置信度: **99.92%**), 1 (0.03%)

- **Q**: "Are there both a cat and a dog in this picture?"
  A: yes (置信度: **99.95%**), no (0.04%)

算法原理分析: 动物计数预测为"2" 的置信度达 **99.92%**，共存确认"yes" 置信度达 **99.95%**。凭借 Transformer 的 Self-Attention 机制，模型成功建立了跨区域的语义关联，聚合了不同位置的视觉特征，从而形成了" 猫和狗同时共存" 的高层语义理解。当猫和狗分别位于图像的不同区域时，模型能够通过 Attention Weight 将两者的视觉特征聚合到同一个语义空间，实现组合逻辑推理。

4.1.3 三、模型的局限性：密集计数灾难（图 6-7）



图 6: 图 6 - 密集目标：多只金毛幼犬同框

- **Q**: "What kind of animal is in this picture?"
  A: dog (73.06%), dogs (21.76%), golden retriever (1.06%)
- **Q**: "How many animals are there in total?"
  A: 6 (22.80%), 8 (22.58%), 7 (21.44%)
  [严重问题] 置信度高度分散，模型彻底丧失计数确定性



图 7: 图 7 - 人宠交互：家庭与宠物合影

- **Q**: "How many people are in the picture?" (实际为 3 人)
  A: 2 (85.43%), 3 (13.83%), 4 (0.59%)
  [错误] 人物计数严重错误
- **Q**: "What kind of animal is in the picture?"
  A: dog (置信度: **99.90%**), dogs (0.04%)

- **Q**: "How many animals are there?" (实际为 1 只狗)
  A: 2 (86.50%), 1 (12.12%), 3 (1.15%)
  [错误] 动物计数错误

查阅资料进行归因分析: 为什么 ViLT 在计数任务上表现如此糟糕？从模型架构设计层面剖析：

- **Patch-based 特征提取的先天缺陷**: ViLT 直接将图像切分成离散的 Patch 序列送入 Transformer，彻底丢失了实例级别（Instance-level）的边界信息。当多个目标高度重叠、颜色纹理相似时，Patch 特征无法有效区分" 这是第 1 只狗" 还是" 这是第 2 只狗"。
- **摒弃区域提议网络（RPN）的代价**: ViLT 追求极速轻量化而摒弃了传统的区域提议网络（如 Faster R-CNN 的 RPN 模块），导致无法有效区分重叠、密集的相似目标。模型缺乏 Object Proposals，无法对每个候选区域进行独立计数。
- **缺乏显式的逻辑计数机制**: ViLT 的分类头直接从预定义的答案词汇表中选择答案，缺乏专门的计数推理模块。模型无法执行" 遍历所有目标实例并累加计数" 的逻辑操作，只能依赖特征的统计相关性进行盲猜。
- **语言先验偏置（Language Bias）作祟**: VQAv2 数据集中，"How many..." 类问题的答案分布呈现明显的长尾偏态——小数字（如 1、2、3）的出现频率远高于大数字。这种数据偏置导致模型在不确定时极其倾向于输出训练集中的高频答案（如数字"2"）。图 7 中将 1 只狗误判为 2 只，正是这种语言偏置的典型体现。

工程启示: ViLT 适用于单目标识别、属性提取、简单多目标共存判断等任务。对于密集目标计数、细粒度实例分割等任务，应考虑采用更先进的架构（如 BLIP-2、LLaVA 等结合大语言模型的 VQA 系统），或引入专门的计数模块。

5 实验总结

5.1 注意事项

1. **网络问题**: 国内直连 Hugging Face 可能失败，需配置镜像源 `HF_ENDPOINT`，首次运行时需要从 Hugging Face Hub 下载模型（约 500MB），请确保网络连接正常
2. **GPU 支持**: 建议使用 GPU 进行推理以获得更快的响应速度
3. **图像格式**: 支持常见图像格式（jpg, png, bmp 等），非 RGB 图像会自动转换
4. **问题设计**: ViLT 主要使用英文预训练，英文问题效果更好；问题应尽量清晰明确

5.2 实验收获

1. **多模态学习**: 理解了视觉-语言多模态模型的基本原理和实现方式
2. **Transformer 架构**: 学习 Transformer 在多模态任务中的应用
3. **Hugging Face**: 掌握使用 Hugging Face Transformers 库加载预训练模型的方法，以及通过镜像源解决网络问题的技巧
4. **工程实践**: 实践模块化代码设计，包括类的封装、错误处理、批量测试等

5.3 参考资料与链接

- ViLT 论文: "ViLT: Vision-and-Language Transformer Without Convolution or Region Supervision"
- Hugging Face 文档: https://huggingface.co/docs/transformers/
- Hugging Face 镜像: https://hf-mirror.com/
- VQAv2 数据集: https://visualqa.org/